ML-POWERED SUPPLY CHAIN

Supply Chain Demand Sensing & Inventory Optimization

Using ML

From Reactive Replenishment to Predictive Intelligence

ML Forecasting Engine | Feature Engineering

Inventory Optimization

PROJECT IDENTITY

Project Overview

Core Mission

Transform static, reactive supply chain planning into a continuously-sensing, ML-driven predictive intelligence system capable of anticipating demand shifts before they occur.

Scope

Retail demand data · Promotional signals · Inventory buffers · Reorder logic

"We don't react to demand — we anticipate it."

Team Members

PIDUGU KALYANI
An undergraduate student in Chemical Engineering at IIT Madras

VEMURI NAVADEEP
An undergraduate student in Chemical Engineering at IIT Madras

JOURNEY MAP

Project Journey: 6-Stage Roadmap

A structured path from raw business problem to measurable supply chain impact.

① Problem Context

Demand variability, stockouts, and overstock driven by reactive forecasting

② Data Understanding

POS sales, promotions, calendar events, SKU-level historical demand

③ Feature Engineering

Lag features, rolling windows, weekday/month encoding, promotion flags

④ ML Modeling

XGBoost regressor · 300 estimators · depth 6 · LR 0.05 · cross-validated

⑤ Forecast Translation

SKU-level demand forecasts converted to safety stock and reorder points

⑥ Impact

MAPE 8.6% · R^2 0.948 · 23% safety stock reduction · improved visibility

Each stage builds on the last — from raw data to actionable inventory intelligence.

PROBLEM & GAP

The Demand Planning Gap

Problem Context

Demand Variability

Unpredictable shifts in consumer behavior cause forecast errors and excess inventory

Promotions Impact

Promotional spikes are poorly captured by static models, causing stockouts

Inventory Imbalance

Simultaneous overstock in slow movers and stockouts in high-velocity SKUs

Key Demand Disruptors

STOCKOUT

OVERSTOCK

VARIABILITY

SEASONALITY

PROMOTIONS

Reactive vs. Predictive

Reactive	Predictive
Past data only	Patterns + signals
Weekly updates	Continuous sensing
High buffer stock	Optimized inventory
Lag in response	Anticipates shifts

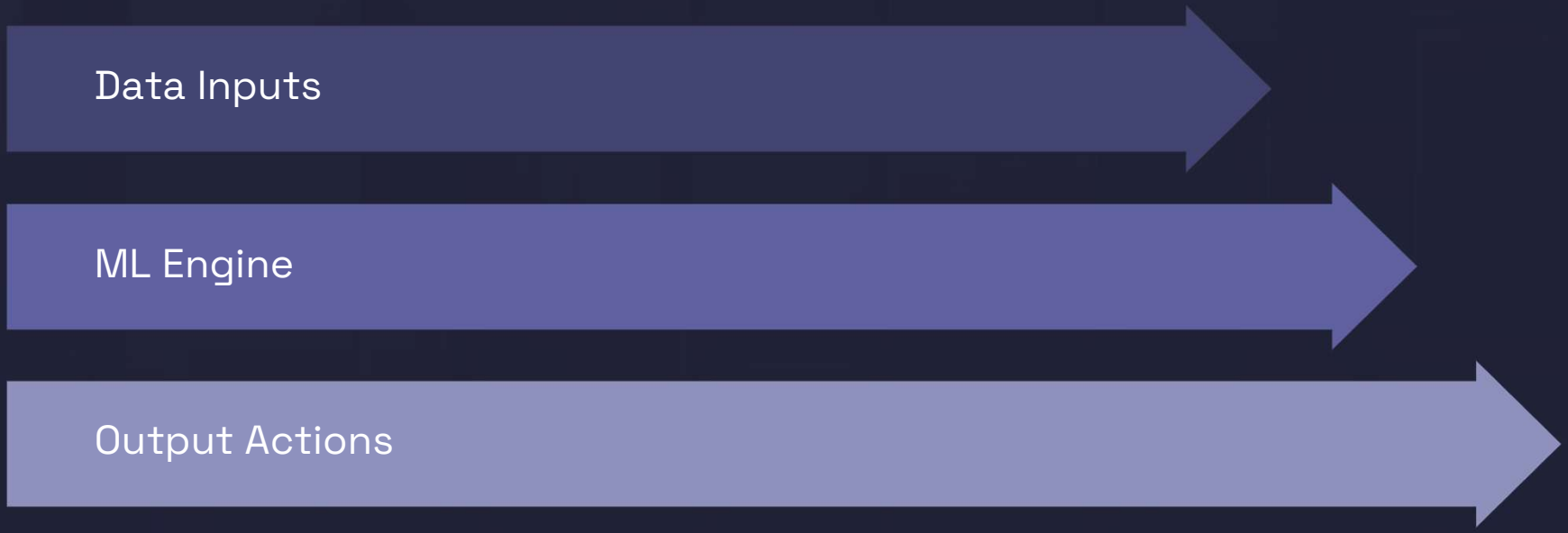
⚠ "Static forecasts fail to capture real-time demand behavior." — This gap costs the business in both excess inventory and lost sales.

SOLUTION ARCHITECTURE

ML Solution Architecture — 3-Layer Design

DATA LAYER

Sales Data — Historical POS transactions at SKU level
Promotions — Discount flags, campaign indicators
Calendar Signals — Weekday, month, holidays, seasonality



ML ENGINE — XGBoost Core

XGBoost Model · n_estimators=300 · max_depth=6 · learning_rate=0.05 I Lag Features: lag_7, lag_14, lag_28 I Rolling Trends: rolling_mean_4w, rolling_std_4w

OUTPUT: Demand Forecast

SKU-level weekly demand prediction with confidence range (e.g., 96 units ± 40)

OUTPUT: Safety Stock

Dynamically computed buffer based on forecast variance and service level targets

OUTPUT: Reorder Point

Automated ROP triggers replenishment before stockout risk window is reached

"We don't react to demand — we anticipate it." I All three layers are connected — data flows up to engine, decisions flow down to inventory.

DEMAND FLOW

End-to-End Demand Intelligence Pipeline

DEMAND SIDE

01

POS Data

Raw point-of-sale transactions ingested at SKU-store level daily

02

Feature Engineering

Lag windows, rolling stats, promo flags, calendar encoding applied

03

Forecast Output

XGBoost produces demand forecast for each SKU at weekly horizon

ML CORE BLOCK

XGBoost

Gradient boosted trees · 300 estimators · Depth 6

Feature Importance

Recent lag features dominate ~60% of model weight

Time Patterns

Weekday, monthly cycles, and seasonal peaks captured

MAPE: 8.6% · R^2: 0.948 · MAE: 5.78

SUPPLY SIDE

01

Safety Stock

Calculated dynamically from forecast variance and service level target

02

Reorder Point

ROP triggered automatically when on-hand stock approaches threshold

03

Inventory Decision

Replenishment order quantity and timing determined from forecast

"Forecast drives inventory decisions in real-time" — closing the loop from POS signal to replenishment action automatically.

MODEL & CODE

Model Implementation & Performance

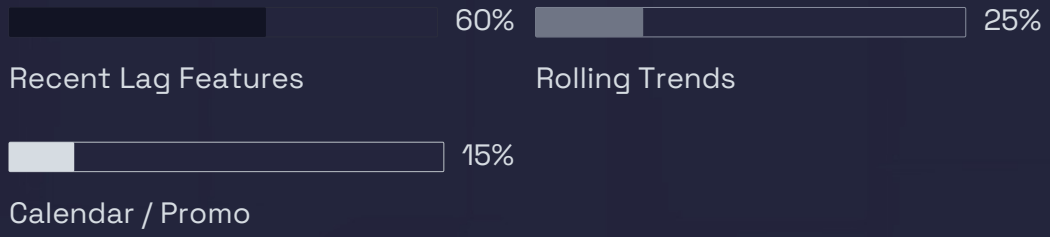
Model Code — XGBoost Implementation

```
features = ['lag_7','lag_14','lag_28',
            'rolling_mean_4w','rolling_std_4w',
            'weekday','month','onpromotion']

model = XGBRegressor(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Feature Importance Insight



Model Performance Metrics

8.6%

MAPE

Mean Absolute Percentage Error — strong accuracy for retail demand

0.948

R^2 Score

Explains 94.8% of variance in demand — excellent predictive power

5.78

MAE (units)

Mean Absolute Error — average forecast deviation of ~6 units per SKU

Forecast Output Example

96 units ± 40

SKU-level weekly forecast with confidence interval. Safety stock derived from the ±40 variance band.

"Model relies heavily on recent demand trends (~60% importance)" — lag_7 and lag_14 are the dominant predictors.

PIPELINE

End-to-End ML Pipeline — 8-Stage Process

Data Collection

Ingest POS sales, promotions, calendar, and inventory data from source systems

Cleaning

Handle missing values, remove outliers, standardize SKU identifiers and date formats

Model Training

Train XGBRegressor with 300 estimators, max_depth=6, learning_rate=0.05 on historical data

Evaluation

Validate on holdout set · MAPE: 8.6% · R^2: 0.948 · MAE: 5.78 units

Feature Engineering

Encode weekday, month, promotion flags, and calendar signals into model-ready features

Lag & Rolling Features

Compute lag_7, lag_14, lag_28, rolling_mean_4w, rolling_std_4w for each SKU

Forecast Generation

Generate SKU-level weekly demand forecasts with confidence intervals (e.g., 96 ± 40 units)

Inventory Translation

Convert forecasts into safety stock levels and reorder points to drive replenishment

"We don't react to demand — we anticipate it." | Each pipeline stage is automated — from raw data ingestion to inventory replenishment trigger.

IMPACT

Business Impact — Dashboard View

Forecast Accuracy Metrics

Forecast Accuracy

Based on MAPE of 8.6% — strong performance for retail demand

Variance Explained

$R^2 = 0.948$ — model captures nearly all demand signal variation

Operational Improvements

Better Demand Visibility

Continuous sensing replaces static weekly batch updates with near-real-time signals

Reduced Overstock

ML-calibrated safety stock eliminates excess buffer inventory across slow-moving SKUs

Fewer Stockout Events

Earlier replenishment triggers prevent lost sales in high-velocity SKUs

🎯 Headline Result

23% ↓

Safety Stock Reduction

Achieved by replacing static buffer rules with ML-derived, variance-calibrated safety stock calculations

📦 Forecast → Inventory Effi ciency

MAPE: 8.6%

Low error = smaller required safety buffer

MAE: 5.78 units

Tight deviation = precise ROP triggers

"Forecast accuracy directly reduces inventory buffers" — every 1% MAPE improvement unlocks further stock reduction.

CLOSING

Thank You

Demand Sensing & Forecasting Using ML

Data

POS · Promotions · Calendar · Inventory signals

Forecast

XGBoost · MAPE : 8.6% · R^2 : 0.948 · MAE : 5.78

Decision

Safety Stock · Reorder Point · 23% buffer reduction

"We don't react to demand — we anticipate it."

From Data → Forecast → Decision